Jeu de Mansuba

Allan DENOCE & Mohamed Aymane KHERRAZ

13 janvier 2023

Table des matières

1 Introduction **3**
1.1 Monde et plateau de jeu 3
1.2 Les pièces et mouvements 4
1.3 Caractéristiques spécifiques au jeu 5

2 Gestion des fichiers et du travail en groupe **5**
2.1 Makefile 5
2.2 Dépendances 6
2.3 Dépôts git 7
2.4 Travail en binome 7

3 Principe du jeu **7**
3.1 Base du monde 7
3.2 Plateau de jeu et relations 7
3.3 Des pièces 9
3.3.1 Les pions classique 10
3.3.2 La tour 10
3.3.3 L'éléphant 11
3.4 Randomisation des coups 11
3.5 Définition des règles du jeu 11

4 Difficultés rencontrées **11**
4.1 Les fichiers 11
4.2 Problème avec le random 12
4.3 Dépendances mal faites 12
4.4 Difficulté à debuger 12

5 Test **13**
5.1 Test global sur le monde 13
5.2 Test des plateaux de jeux 13
5.3 Test des sauts 13
5.4 Test sur la tour et l'éléphant 14
5.5 Test sur le déroulé de la partie 14

6 Conclusion **14**
6.1 Vue globale 14
6.2 Ouverture sur le vrai monde de jeux 14

7 Bibliographie **15**

1 Introduction

Le sujet du projet ce base sur la création d'un jeu de mansuba. Pour se faire, il a fallu créer le jeu depuis sa base jusqu'au moment où il pouvait jouer automatiquement sans aucune intervention extérieur.

1.1 Monde et plateau de jeu

Notre jeu est composé d'un monde qui définit l'ensemble des positions qu'on pourra utiliser pour jouer. Ces positions correspondent aux différentes cases qu'on peut trouver sur un plateau de jeu classique. Ce monde peut être modifié notamment au niveau du nombre de colonnes et de lignes qu'on peut trouver. Les valeurs du plateau de base est un monde de largeur égale à 5 et de hauteur égale à 4.

Avec ce monde, on a aussi créé différentes relations entre les positions du monde afin de pouvoir définir notre plateau de jeu. Dans notre jeu, on peut utiliser 3 plateaux de jeux différents : Un plateau carré où chaque position a 8 voisins (Les 4 positions sur ses côtés et les 4 sommets du carré), un plateau de jeu triangle qui lui a seulement 3 voisins correspondant à ses 3 côtés et un plateau de jeu hexagonale avec 6 voisins qui sont pour lui aussi au niveau de ses côtés. Pour rendre les parties plus intéressantes, il y a la possibilité de changer de plateau de jeu en plein milieu de la partie.

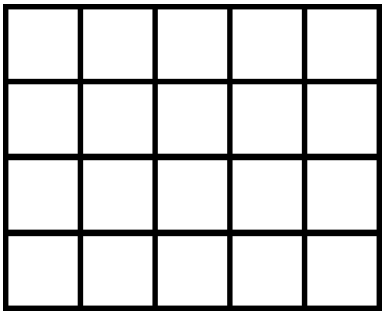

Figure 1 – Plateau carré théorique

Figure 2 – Plateau carré réel

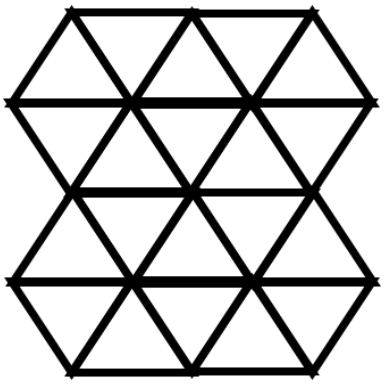

Figure 3 – Plateau triangle théorique

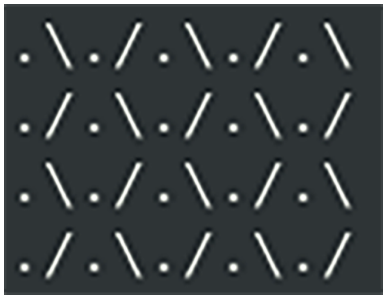

Figure 4 – Plateau triangle réel

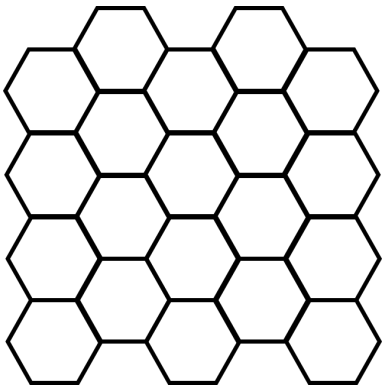

Figure 5 – Plateau hexagonale théorique

Figure 6 – Plateau hexagonale réel

1.2 Les pièces et mouvements

Ensuite, on a 3 types de pièces dans notre jeu de dames qui peuvent faire des mouvements différents. On a le pion simple qui peut effectuer 3 mouvements différents. Il peut faire un déplacement simple ce qui correspond à se déplacer sur une case voisine qui est inoccupée, il peut aussi faire un saut simple ce qui correspond au fait de sauter par dessus une autre pièce lui permettant de parcourir 2 cases et le saut multiple qui correspond à une succession de saut simple. Pour les 2 types de sauts, le pion peut aussi bien sauter des pions de sa couleur que des pions de la couleur adverse. On retrouve aussi la tour parmi les pièces présentes dans le jeu, elle peut se déplacer uniquement dans l'une des 4 directions cardinales mais elle n'est pas limité par le nombre de cases parcouruent

tant qu'elle ne rencontre pas un autre pion. Pour finir, on a l'éléphant qui lui peut se déplacer de 2 cases (L'une après l'autre) dans les directions cardinales. Il peut donc avancer de 2 cases dans les directions cardinales ou d'une seule dans les diagonales. Le choix de la pièce qui se déplace et le déplacement qu'elle effectue est choisit de manière aléatoire.

1.3 Caractéristiques spécifiques au jeu

Au début de la partie, les pièces du joueur noir se trouve sur la première colonne tandis que les pièces du joueur blanc se trouve sur la dernière colonne. Les 2 joueurs doivent avoir au début de la partie le même nombre de pièces et ces pièces doivent être placé de manière symétrique.

Afin de remporter la partie, les joueurs doivent avoir une de leurs pièces qui atteint la position de départ de l'adversaire, on appelle ceci une victoire simple. Une deuxième condition de victoire est la victoire complexe qui correspond au fait d'avoir toutes ses pièces sur toutes les positions de départ de l'adversaire. Le nombre de tour maximum est configuré de base pour être 2 fois la taille du monde. Avec cette taille, il est très dur d'atteindre une victoire complexe c'est pourquoi il y a la possibilité d'augmenter le nombre de tour maximum afin que cette condition de victoire soit réalisable.

2 Gestion des fichiers et du travail en groupe

2.1 Makefile

Le Makefile nous a permis d'automatiser des actions qui sans ce dernier auraient pris beaucoup de temps. Dans ce projet le Makefile à été utilisé pour automatiser la compilation de fichiers .c en fichiers .o puis la combinaison de fichiers .o pour créer des exécutables (projet, test, ...) qui servent à jouer le jeux et faire différents tests de fonctions.
Nous avons rencontré un problème au cours de la création de ce fichier car les fichiers qui ont pour tache de tester les fonctions (dans ./tst) n'ont pas toutes les signatures et les bibliothèques nécessaires pour fonctionner correctement. Pour résoudre ce problème, on été obligé de faire une copie de tous les fichiers .h dans le répertoire ./tst/ et on les supprime par la suite avec tous les fichiers .o qui ont permis de construire l'exécutable.(On les ajoute et les supprime à chaque fois pour qu'il reste à jour avec toutes les modification qu'on fait dans les fichiers .h originaux qui se trouvent dans le répertoire ./src).

```
test.o: tst/test.c
    cp -r ./src/*.h ./tst
    $(CC) $(CFLAGS) tst/test.c -c -o test.o

test_show_world.o: tst/test_show_world.c
```

```
    cp -r ./src/*.h ./tst
    $(CC) $(CFLAGS) tst/test_show_world.c -c -o test_show_world.o

clean:
    rm -f *.o *~ project test test_show_world
    rm -r ./tst/*.h
```

2.2 Dépendances

Dans le graphe de dépendance les fichiers appelés source.c et test.c correspondent à tout les fichiers .c présent dans les répértoires src et tst. On retrouve notamment pawn.c, tower.c, rules.c ...

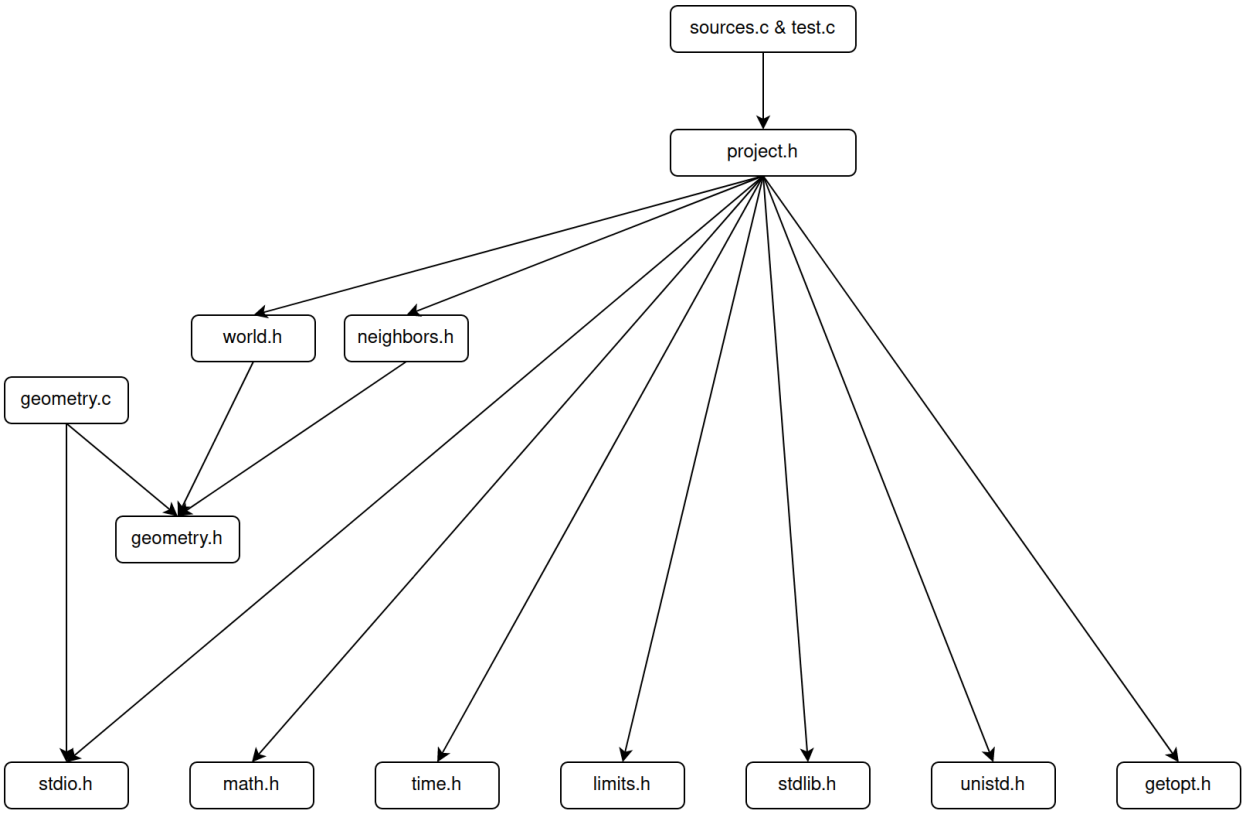

FIGURE 7 – Graphe de dépendance

2.3 Dépôts git

Nous avons utilisé un dépôt git tout au long du projet. Ce dépôt a été un outil important notamment pour le travail en groupe car celui-ci permettait de déposer des bouts de codes et de récupérer les parties de son binôme. Pour se faire il a fallu commit notre code depuis le terminal de l'ordinateur. A certains moments, des conflits apparaissaient car nous avions modifié les mêmes parties du code. A ce moment là, il a fallu se concerter afin de déterminer ce que l'on devait garder et ce que l'on devait enlever.

2.4 Travail en binome

Le travail en binome a été une expérience intéressante et enrichissante car jusqu'à maintenant nous n'avions jamais été amené à travailler en groupe pour un projet de programmation. Coder à plusieurs est particulier car il est parfois nécessaire d'utiliser ou de modifier le code de son partenaire, ce qui fait qu'on est obligé de faire un code clair et de commenter les parties les plus complexes afin qu'il puisse comprendre comment la fonction marche. Le travail en groupe c'est plutôt bien passé car on était souvent sur la même longueur bien que parfois on était pas spécialement d'accord sur la façon de faire certaines fonctions.

3 Principe du jeu

3.1 Base du monde

Initialisation du monde : Le monde est une structure qui contient un tableau de la taille du monde et qui renseigne pour chaque indice (Chaque indice correspond à une case), la couleur et le type du pion qui est sur cette case. L'initialisation du monde consiste à mettre dans notre structure monde, pour chaque indice, pas de couleur (NO_COLOR) et pas de pion (NO_SORT).

Création des fonctions qui agissent sur le monde : On a ensuite créé différentes fonctions qui interagissent avec le monde. On a les fonctions world_get et world_get_sort qui permettent de connaître la couleur et le type de pion pour un indice donné dans notre monde. On a de même les fonctions world_set et world_set_sort qui permettent de changer la couleur et le type de pion d'un indice dans le monde. C'est à l'aide des fonctions set qu'on va initialiser le plateau de jeu en plaçant les pions noirs sur la première colonne et les pions blancs sur la dernière colonne grâce à la fonction position_init.

3.2 Plateau de jeu et relations

Dans notre jeu de dames, on a créé 3 plateaux de jeu : le carré, le triangle et l'hexagone. Les 3 plateaux de jeu sont définis de la même manière, les indices vont de manière croissante de la gauche vers la droite et du haut vers le bas. Le fait que les indices des cases soit similaire entre les plateaux nous permet de

changer de plateaux au cours de la partie de manière simplifiée. On change de plateau toutes les racines de MAX_TURN où l'on change les relations selon le plateau de jeu grâce à la fonction get_neighbor_in_table.

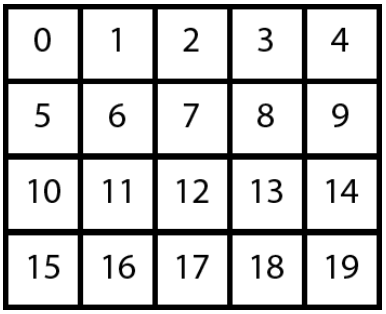

FIGURE 8 – Plateau carré avec les indices

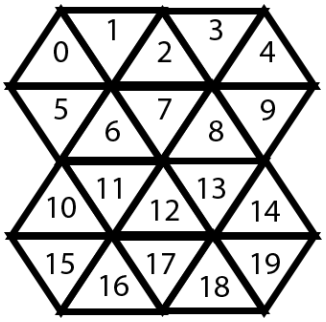

FIGURE 9 – Plateau triangle avec les indices

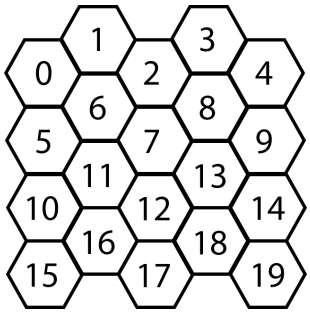

FIGURE 10 – Plateau triangle avec les indices

Ce qui change principalement entre les 3 est la définition de ses voisins (la fonction get_neighbor). Pour le plateau carré, chaque case à 8 voisins : Les 4 carrés sur les côtés du carré et les 4 au niveau des sommets. Il a fallu définir des cas limites pour toutes les cases qui se trouvent au niveau des bords du plateau afin de rendre inaccessible les cases en dehors du plateau de jeu. Les cas limites sont les mêmes pour le carré, l'hexagone et le triangle. Pour le plateau avec les cases hexagonales, on a définit 2 types d'hexagones, les hexagones "haut" (indice 6 dans la figure 11) et les hexagones "bas" (indice 7 dans la figure 12). Par exemple sur la figure 11, les voisins de 6 sur la même ligne (5 et 7) sont respectivement à l'ouest et à l'est. Il a 3 voisins au Nord et 1 voisin au Sud. On prend donc pas en compte les directions Sud Est et Sud Ouest. Pour les hexagones en position basse c'est l'inverse, se sont les directions Nord Est et Nord Ouest qui ne sont pas pris en compte.

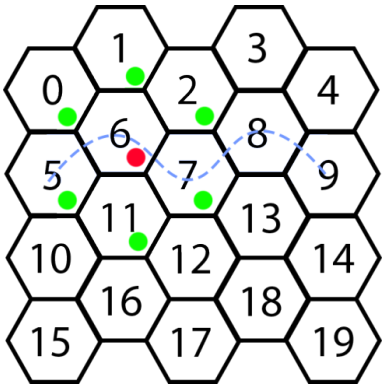

FIGURE 11 – Hexagone en position haute

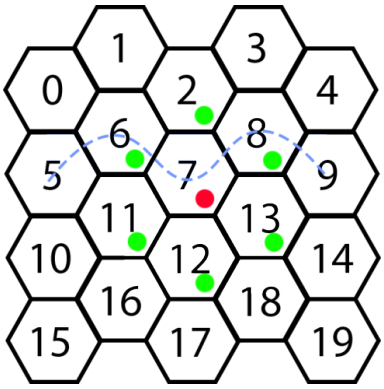

FIGURE 12 – Hexagone en position basse

Le triangle est définie de la même manière mais lui n'a que 3 voisins, Le triangle à l'Est, à l'Ouest et au Sud si il a la "tête en haut" voir figure 13 (au Nord si il a la "tête en bas" voir figure 14).

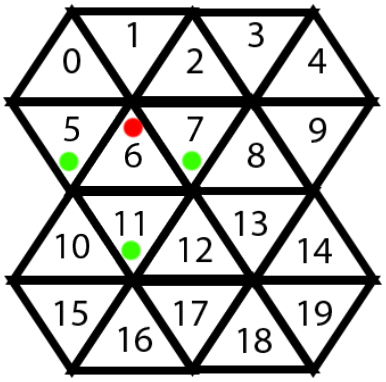

FIGURE 13 – Triangle pointant vers le haut

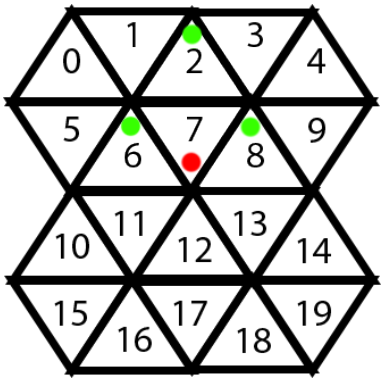

FIGURE 14 – Triangle pointant vers le bas

On a aussi une fonction get_neighbors qui renvoie tout les voisins d'une case et leur direction grâce aux relations vu précédemment.

3.3 Des pièces

Comme nous nous sommes réparti les différentes fonctions liées aux pièces, elles ne sont pas toutes définies de la même manière. Chaque pièce a une fonction move qui permet d'obtenir l'indice d'arrivée de la pièce.

3.3.1 Les pions classique

Pour les pions classique, il y a 3 mouvements possible. D'abord le déplacement simple qui prend en argument le monde et l'indice du pion et qui renvoie la structure neighbors avec les indices de toutes les cases vides autour de lui ainsi qu'un UINT_MAX à la fin des cases disponibles servant de délimiteur.

Le saut simple a été construit sur le même principe, On regarde les différents pions qui sont voisins à notre pion en train de jouer et si dans la même direction, la case d'après est libre. Tout les pions possibles à sauter voient leur indice stocké dans la structure neighbors. Dans la structure neighbors, on stock les indices des pions qu'on peut sauter et non la position d'arrivée après le saut. La fonction move_pawn s'occupera de renvoyer l'indice d'arrivé.

Pour le saut multiple on a décidé de faire un peu différemment car on renvoie à la fin de la fonction directement l'indice d'arrivée. Pour ce qui est de la fonction, on effectue à chaque fois le premier saut simple de notre fonction et on place un pion à la position après le saut, on laisse le pion à la position de départ afin de ne pas pouvoir revenir sur nos pas (Représenté par les étapes de 1 à 3 de la figure 15). Ensuite, on nettoie toutes les pièces qu'on a ajouté dans la fonction (représenté par les points rouges à l'étape 4) et on renvoie l'indice d'arrivée (point vert dans la figure 15). On est plutôt content de ce qu'on a fait mais pas entièrement satisfait car on prends toujours le premier saut qui nous viens et pas un choix aléatoire selon tout les sauts possibles et que notre fonction nous renvoie un seul saut multiple possible et non l'ensemble des sauts réalisables ce qui diminue la partie aléatoire.

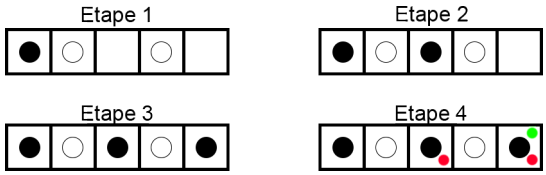

FIGURE 15 – Algorithme de saut multiple

3.3.2 La tour

Le déplacement de la tour est similaire à celui d'un simple pion, on regarde si les 4 cases sont libres dans les 4 directions cardinales. Dans la fonction move_tower on va choisir une direction aléatoire disponible et ensuite on va se déplacer aléatoirement sur l'une des cases libres qu'on trouve dans cette direction avant de rencontré un pion.

3.3.3 L'éléphant

Pour l'éléphant il fallait parcourir toutes les directions cardinales et puis parcourir les directions cardinales du voisin en supprimant le retour en arrière (get_neighbors_elephant), pour choisir un mouvement aléatoire il fallait éliminer toutes les case non vides (simple_movement_elephant) et compter le nombres de mouvements qu'il peut faire (ce calcul est fait par la fonction number_semidiag).

3.4 Randomisation des coups

Afin de faire un mouvement aléatoire, on a décomposé l'aléatoire en 2 parties. En premier on choisit une pièce du joueur en train de jouer de manière aléatoire, ensuite on choisit aussi de manière aléatoire l'un des mouvements possible pour cette pièce.

Pour choisir une pièce aléatoirement, on parcourt l'intégralité du monde afin de compter le nombre de pièces dont on dispose. On choisit ensuit une pièce aléatoirement avec la fonction rand() puis on finit par parcourir de nouveau le monde jusqu'à atteindre la pièce choisit où l'on retourne son indice. On est pas satisfait de la complexité de cet algorithme car on parcours le monde en entier alors qu'on aurait juste pu stocker les positions des pièces dans une structure.

Ensuite, l'indice retourné est utilisé dans la fonction pour choisir un mouvement aléatoire (choose_random_move_for_piece). Cette fonction regarde juste le type du pion et renvoie la fonction move associé au pion.

3.5 Définition des règles du jeu

Pour l'algorithme des conditions de victoire, il est séparé en 2 parties : la victoire simple et la victoire complexe. Pour la victoire simple, l'algortihme regarde uniquement si un pion blanc ou noir a atteint le côté adverse en divisant son indice par la largeur du plateau. Pour ce qui est de la victoire complexe, on se base sur le même principe mais on utilise une boucle et un compteur afin de voir si tout les pions ont atteint l'autre côté.

4 Difficultés rencontrées

Tout au long du projet, on a été confronté à différents problèmes qu'on a du résoudre. Dans cette section on va décrire les problèmes rencontrés ainsi que les solutions qu'on a mis en place afin de les résoudre.

4.1 Les fichiers

Au début du projet, on inséré les différents bouts de codes dans les fichiers *neighbors.c et neighbors.h* et *world.c et world.h* ce qui a posé problème par la suite car la compilation dans la forge ce faisait avec les fichiers .h de la base

du jeu et non les notres. On a par la suite stocké toutes nos fonctions dans un même .c ce qui posé beaucoup de problèmes de clarté car on avait plus de 450 lignes avec des fonctions qui n'étaient pas écrites dans un ordre logique. On a donc fini par créer des fichiers .c en rapport avec chaque élément de notre jeu. Par exemple, on a un .c pour chaque pièce (avec ses mouvements) et un .c par plateau de jeu.

4.2 Problème avec le random

La seed de la fonction rand (fonction srand) prend en paramètre la fonction time qui nous donne l'heure en seconde afin d'avoir des seed différentes. On s'est rendu compte que comme le code s'applique de manière quasiment instantanée pour la boucle de jeu, la valeur renvoyé par la fonction rand ne varié pas et on se retrouvait souvent avec uniquement 2 pièces (1 par couleur) qui jouaient, et les pièces étaient miroir l'une de l'autre. Pour remédier à ce problème, on a décidé de mettre un sleep d'1 seconde dans notre boucle de jeu ce qui permet d'avoir la fonction time qui change à chaque fois qu'on l'appelle srand et donc d'avoir un jeu plus aléatoire. On est pas totalement satisfait de cette solution car notamment pour les parties avec victoire complexe, le jeu peut prendre plusieurs heures.

FIGURE 16 – Sans sleep

FIGURE 17 – Avec sleep

4.3 Dépendances mal faites

Au début de notre projet, les dépendances étaient une notion très floue, ce qui nous a conduit à créer une double dépendance entre deux fichiers. C'est à dire que fichier A contient #include B et fichier B contenait #include A. En dépit de notre faible compréhension de ces concepts, nous avons été en mesure de corriger l'erreur très rapidement.

4.4 Difficulté à debuger

Dans certaines fonction nous n'avions pas traité le cas d'indice egale à UINT_MAX et souvent les fonctions {get_sort, get, set ,set_sort} retournaient des erreurs oc-

casionnelles avec des valeurs d'indices bizarres (=4294967295) après un peu de recherche on a été capable de remarquer que le nombre là est UINT_MAX. La question qui se posait est : d'où vient ce problème la réponse à cette question nous a pris un peu de temps car on connais pas l'existence du débugger GDB. C'est un outil très fort pour voir les détails d'exécution du code.

5 Test

Les tests sont très importants afin de s'assurer de la fiabilité de nos différents code. Dans cette section, on va présenter les différents tests qu'on a réalisé au cours du projet. En plus des tests effectués ci-dessous, on a lancé des parties de jeux en affichant le plateau ce qui nous a permis de déceler plusieurs problèmes dans notre code.

5.1 Test global sur le monde

Pour tester toutes les fonctions en rapport avec le monde, on a créé un monde de taille 100 (10x10) où on a initialisé les positions de départ des pions. On a ensuite affiché la couleur et le type de pion pour chaque indice du tableau et vérifier qu'on a bien pas de pion et de couleur pour toutes les cases sauf celles des indices 0,10,20,.. qui doivent être noir avec un pion et les indices 9, 19, 29,... qui doivent être blanche avec un pion.

5.2 Test des plateaux de jeux

Afin de pouvoir tester les différentes relations qu'on a mis en place, on a créé des fonctions show_world qui permettent d'afficher les différents plateaux de jeu. En lançant différentes parties, on a pu vérifier grâce à l'affichage que les pièces se déplacent bien comme on l'avait prévu. Ça nous a aussi permis de trouver différents problèmes dans la définition de nos fonctions notamment l'oubli de certains cas limites qui nous donnait des pions qui passaient du bord gauche au bord droit du plateau en 1 seul coup.

5.3 Test des sauts

Pour tester les fonctions saut simple et sauts multiples, on a décidé de réduire la taille du plateau de jeu afin qu'il ne reste plus qu'une seule ligne pour forcer les pions à effectuer les sauts. Si un joueur gagne, alors le saut simple marche. Pour le saut multiple, on a décidé d'ajouter manuellement des pions 1 cases sur 2 en laissant la dernière case vide. On a forcé le joueur avec 1 seul pion a commencé à jouer et on l'a empêché de faire un saut simple. Si il gagne du 1er coup, alors le saut simple marche. Ce test est illustré par la figure 15.

5.4 Test sur la tour et l'éléphant

Pour vérifier que l'éléphant et la tour avaient bien les comportements attendu, on a créé des plateaux de jeux avec uniquement des tours et des éléphants et on a regardé si le comportement des pions était bien celui qu'on attendait.

5.5 Test sur le déroulé de la partie

Différentes fonctions nécessaires pour le bon déroulé de la partie ont été testé tout du long du projet lorsque l'on simulé des parties de jeux. On retrouve dans ce cas les différentes fonctions désignant le premier joueur, ainsi que le changement de joueur à chaque tour mais aussi le bon fonctionnement des conditions de victoire qu'elle soit simple ou bien complexe. Pour la victoire complexe, nous avons augmenté le nombre de tour maximum à plus de 100 000 afin d'être sûr de pouvoir obtenir une fin de partie.

6 Conclusion

6.1 Vue globale

On a bien réussi à construire le jeu et à manipuler les différents type de variables structures et enum ainsi que la gestion des fichiers et des dépendences. Nous avons été confrontés à de plusieurs problèmes mais nous avons pu trouver des solutions altérnatives pour les résoudre. On peut aussi ajouter que maintenant qu'on a une vision globale sur le sujet on a plus d'idée d'optimisation et d'amélioration.

6.2 Ouverture sur le vrai monde de jeux

Le type de jeu qu'on nous a proposé est un jeu non coopératifs (chacun pour soi) et séquentiel (le mouvement choisi dépend des mouvements des autres joueurs et il y a une récompense à la fin du jeu), exactement comme le célèbre jeu d'échecs d'après les définitions fournies dans document [1]. L'idée naïve serais de crée l'arbre de toutes les parties possible mais ceci est mauvais à cause de sa complexité exponentielle, c'est pour cela qu'on va plutôt utiliser des algorithmes comme celui de Monte-Carlo, min-max ... [2] pour essayer de crée réellement un programme où les deux joueurs veulent gagner.

7 Bibliographie

[1] https ://fr.wikipedia.org/wiki/Théorie_des_jeux

[2] https ://philippmuens.com/minimax-and-mcts